MODULE 16 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

Configuration Management

CLAUDE.md and settings.json reviewed like code, model version pinning, prompt versioning and rollout, feature flags, downstream impact, and plugins with their dependencies.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 16 of 36

admin@sthithapragna.com

The company handbook, the team wiki, the note on your desk. **Each layer adds detail. None of them locks a door.**

The CLAUDE.md hierarchy

LAYER	WHERE	FOR
Managed policy	Deployed by administrators	The whole organisation
User	~/.claude/CLAUDE.md	You, in every project
Project	./CLAUDE.md or ./.claude/CLAUDE.md	Everyone in this repository
Local	CLAUDE.local.md, git-ignored	You, in this repository
Subdirectory	CLAUDE.md in a folder	Loaded on demand, when Claude works there

ON THE EXAM

The files are "concatenated into context rather than overriding". And CLAUDE.md is "context, not enforced configuration".

Writing CLAUDE.md well

- **Short and specific:** build and test commands, conventions, where things live, what to avoid and why.
- **Import rather than copy:** @path imports, up to four hops deep.
- **Scoped rules:** .claude/rules/*.md, with paths: so a rule loads only for matching files.
- **Start with /init,** then edit: it drafts a CLAUDE.md, or suggests improvements to yours.
- **Committed and reviewed** like code, because it shapes every session.

> THE TAKEAWAY
>
> CLAUDE.md is re-read every request, so every line costs context. Keep only what earns its place.

settings.json: scopes and precedence

SCOPE	FILE	TYPICAL CONTENTS
Managed	Deployed by administrators	Organisation locks; cannot be overridden
Command line	Flags for one session	Temporary overrides
Project local	.claude/settings.local.json	Your own tweaks, not committed
Project shared	.claude/settings.json	Team permissions, hooks, plugins, env
User	~/.claude/settings.json	Your defaults everywhere

ON THE EXAM

Higher rows win. Lists such as permission rules merge across scopes instead of overriding.

CLAUDE.md or settings.json?

CLAUDE.MD

Guidance

- Conventions and context
- How we build and test here
- Read by the model, followed as instructions
- "A request, not a guarantee"

SETTINGS.JSON

Enforcement

- Permission allow and deny rules
- Hooks that run every time
- Enabled plugins, environment, model
- Applied by Claude Code itself

THE TIEBREAKER

If it must never happen, it belongs in settings or a hook. Rule one: code enforces, prompts guide.

Model version pinning

- **From Claude 4.6 onward, model IDs are dateless snapshots:** "not an alias. It is the snapshot".
- **Anthropic never changes an existing ID.** A new model gets a new ID.
- **Older models:** pin the dated snapshot ID, not an alias that moves.
- **Deprecations** come with at least 60 days' notice. Sonnet 4.5 retires on 30 November 2026.
- **The ID lives in configuration,** so an upgrade is a reviewed change, not a code hunt.

> THE TIEBREAKER
>
> Pinning stops surprise changes. It does not stop retirement, so plan upgrades before the deadline.

Configuration out of code

```yaml
# config/triage.yaml - reviewed like code, deployed by the pipeline
model: claude-sonnet-5-5        # a snapshot ID: it never changes under you
prompt: prompts/triage/v7.md
prompt_version: 7
effort: medium
max_tokens: 2048
flags:
  draft_replies: true           # turn off without a deploy
  prompt_v8_percent: 5          # canary: 5% of tickets try v8
schema_version: 2               # consumers check this before reading
```

2, 4 Model and prompt pinned, by ID and by version.

9 A flag routes a small share of traffic to the new prompt.

THE TAKEAWAY

Code reads configuration. Configuration changes go through review, evals and rollout, like code.

Rolling out a prompt version

01
Change
New version file, pull request, changelog line.

02
Evaluate
Regression set: compare with the current version.

03
Canary
A flag sends a small share of traffic to it.

04
Expand
Watch metrics, then raise the share.

05
Roll back
The old version stays; one flag change reverts.

ON THE EXAM

Never edit the live version in place. Add a version, prove it, then switch.

Before you change configuration

- **Who consumes it?** Services, reports, agents and people downstream of the output.
- **What changes for them?** Field names, formats, labels, latency, cost.
- **Run their contract tests,** not only your own evals.
- **Version the contract,** and keep the old shape during a transition.
- **Stage the rollout,** with a rollback ready.

> THE TAKEAWAY
>
> A model, prompt or schema change is a change to every consumer of the output.

Plugins in Claude Code

- **A plugin bundles** skills, commands, agents, hooks, MCP servers, LSP config and output styles.
- **A marketplace** is a repository with .claude-plugin/marketplace.json listing plugins and sources.
- **Teams enable plugins** in the shared project settings, under enabledPlugins.
- **dependencies:** "Plugins that must be enabled for this one to work".
- **Pin git sources** to a ref or SHA; administrators can require a marketplace on every machine.

> THE TIEBREAKER
>
> Share Claude Code capabilities as a versioned plugin, not as files copied between repositories.

Copperlet: CLAUDE.md copied everywhere

BEFORE

35 drifting copies

- 300-line CLAUDE.md in 35 repositories
- Already drifted into different versions
- Rules mixed with guesses about enforcement
- Every fix pasted 35 times, or not

AFTER

Layers with owners

- Managed CLAUDE.md: company-wide essentials
- copperlet-eng plugin: shared skills, agents, hooks
- Each repo: a short CLAUDE.md of its own
- Must-never rules in managed settings

SETTLED

Quirk 16 settled: one owner per layer, versioned, nothing pasted.

> A team's project CLAUDE.md says: "Never edit files in the secrets directory." Occasionally, Claude Code still edits a file there during long sessions.
>
> The team needs this to be guaranteed.

What should the team do?

A Repeat the instruction in capitals at the top and bottom of CLAUDE.md

B Move the instruction into each developer's CLAUDE.local.md file

C Enforce it with a deny rule or hook in shared settings, keeping the line as context

D Move the instruction into each developer's user-level CLAUDE.md

◆ PAUSE AND DECIDE

What should the team do?

A Repeat the instruction in capitals at the top and bottom of CLAUDE.md

Louder guidance is still guidance.

B Move the instruction into each developer's CLAUDE.local.md file

Still context, and now unshared.

C Enforce it with a deny rule or hook in shared settings, keeping the line as context

Enforced by Claude Code, for everyone.

D Move the instruction into each developer's user-level CLAUDE.md

A different layer of the same request.

THE SIGNAL

"Guaranteed" means enforcement: settings or a hook, never CLAUDE.md alone.

A service calls an older Claude model through an alias that always points to that family's newest version. After the alias moved, output formats changed in production without any code change.

The team wants model changes to be deliberate.

What should the team do?

A Pin a specific snapshot ID in configuration, and upgrade deliberately after running the evals

B Remove the model setting, and let the SDK choose a default

C Add instructions to the prompt to keep the old output format

D Switch to whichever model is cheapest each month

◆ PAUSE AND DECIDE

What should the team do?

A Pin a specific snapshot ID in configuration, and upgrade deliberately after running the evals

Changes happen only when you choose.

B Remove the model setting, and let the SDK choose a default

Less control, not more.

C Add instructions to the prompt to keep the old output format

Patches one symptom of an unpinned model.

D Switch to whichever model is cheapest each month

Scheduled surprise.

CURRENCY CHECK

Pin snapshots. From Claude 4.6 on, every model ID is already a snapshot that never changes.

A team has written a new version of a customer-facing prompt. The current version serves all traffic. The last prompt change caused an incident that took hours to undo.

The team wants to release the new version with less risk.

Which two practices should they follow? Choose two.

A. Edit the current version in place in production, so there is only one prompt to manage

B. Run the regression set on the new version and compare it with the current version before release

C. Delete the current version once the new version is live, to avoid confusion

D. Stop logging the prompt version, to keep the logs small

E. Release behind a flag to a small share of traffic, keeping the current version for instant rollback

◆ PAUSE AND DECIDE

Which two practices should they follow? Choose two.

A Edit the current version in place in production, so there is only one prompt to manage

No comparison, no rollback.

B Run the regression set on the new version and compare it with the current version before release

Evidence before release.

C Delete the current version once the new version is live, to avoid confusion

Removes the rollback.

D Stop logging the prompt version, to keep the logs small

Loses traceability.

E Release behind a flag to a small share of traffic, keeping the current version for instant rollback

Small blast radius, fast undo.

THE SIGNAL

"Took hours to undo" is the signal: evaluate first, then a canary with the old version kept.

> A team renames a field in its extraction schema from "rent" to "monthly_rent" and ships it. The team's own evals pass, but a finance reporting service that reads the output breaks overnight.
>
> The team wants to avoid this next time.

What practice should they adopt?

A. Never change the schema again once it has shipped

B. Announce schema changes in release notes, and expect consumers to adapt

C. Rename the field back, and leave the schema as it was

D. Check consumers and their contract tests first, and version the schema with a transition

◆ PAUSE AND DECIDE

What practice should they adopt?

A Never change the schema again once it has shipped

Freezes the product to avoid a process.

B Announce schema changes in release notes, and expect consumers to adapt

Hopes someone reads them in time.

C Rename the field back, and leave the schema as it was

Fixes today, not next time.

D Check consumers and their contract tests first, and version the schema with a transition

Assesses downstream impact before the change.

THE SIGNAL

"Our evals pass, theirs broke": assess downstream impact, not only your own tests.

> Engineering wants the same code-review skills, subagents and safety hooks in all 35 repositories. Teams have been copying the files in by hand, and the copies have drifted apart.
>
> Some hooks also rely on a shared MCP server.

What is the best approach?

A Copy the files into every repository with a weekly script

B Paste all the skills and hooks as text into one long shared CLAUDE.md

C Package them as a versioned plugin, with dependencies, from an internal marketplace

D Publish a wiki page and ask each developer to install them by hand

◆ PAUSE AND DECIDE

What is the best approach?

A Copy the files into every repository with a weekly script

Automated drift.

B Paste all the skills and hooks as text into one long shared CLAUDE.md

Text does not run hooks or load skills.

C Package them as a versioned plugin, with dependencies, from an internal marketplace

One source, versioned, with what it needs.

D Publish a wiki page and ask each developer to install them by hand

Manual, uneven, unversioned.

THE SIGNAL

Shared skills, agents, hooks and MCP across repositories: that is what plugins are for.

> A company must make sure no developer uses a particular model in Claude Code, whatever they set in their own settings or environment.
>
> The platform team administers managed settings.

What should the platform team do?

A Add a line to every project CLAUDE.md saying the model must not be used

B Restrict models in managed settings with availableModels or deniedModels

C Set a different model with the model key in each shared project settings file

D Set a different model with the model key in managed settings

◆ PAUSE AND DECIDE

What should the platform team do?

A Add a line to every project CLAUDE.md saying the model must not be used

Context, not enforcement.

B Restrict models in managed settings with availableModels or deniedModels

The documented lock.

C Set a different model with the model key in each shared project settings file

A default developers can override.

D Set a different model with the model key in managed settings

"A starting default, not a lock."

CURRENCY CHECK

A managed model key is "a starting default, not a lock; the locks are availableModels and deniedModels".

Module 16 in five lines

- **CLAUDE.md layers** are concatenated context; settings.json enforces, highest scope wins.
- **Must never happen?** Settings or a hook, not CLAUDE.md.
- **Pin snapshot IDs** in config; upgrade deliberately before retirement.
- **Prompt versions:** add, evaluate, canary, expand, keep the rollback.
- **Check downstream impact;** share Claude Code tooling as versioned plugins.

COMING UP

Next: the systems life cycle - ship, run, change and retire.